

## BÀI 2 - CÀI ĐẶT, KIỂM TRA DOCKER VÀ CHẠY HELLO-WORLD

Tài liệu thực hành có sử dụng ảnh chụp trực tiếp từ máy học viên để đối chiếu kết quả.

### Mục tiêu bài học

- Kiểm tra WSL 2 và xác nhận môi trường Windows đã sẵn sàng cho Docker Desktop.
- Cài và khởi động Docker Desktop.
- Kiểm tra Docker CLI và Docker Engine bằng các lệnh cơ bản.
- Chạy container đầu tiên bằng image hello-world.
- Hiểu luồng thực tế: Docker CLI → Docker Engine → Image → Container → Process.

Phạm vi: Bài này chưa đi sâu vào `docker pull`, `docker ps`, `docker stop`, `docker rm`. Các lệnh quản lý Image và vòng đời Container sẽ học ở Bài 3.

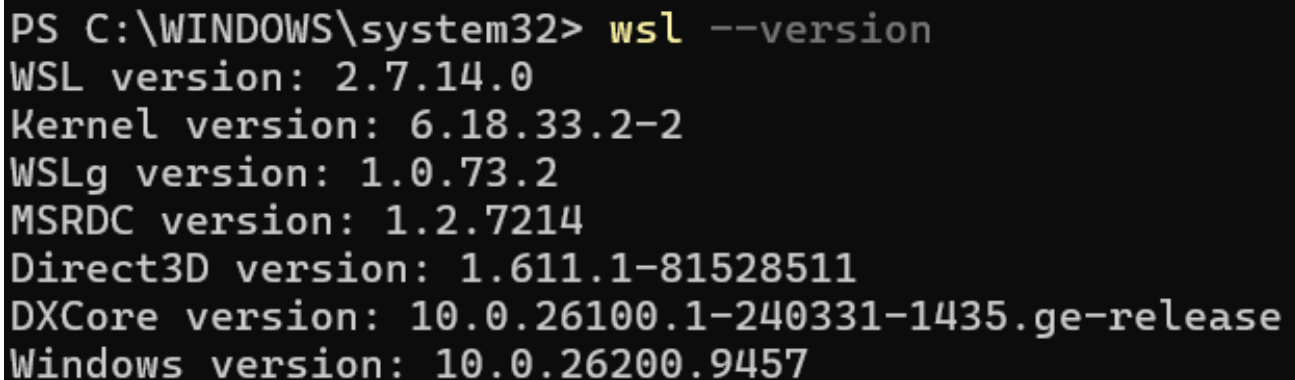
## 1. Kiểm tra WSL 2

WSL (Windows Subsystem for Linux) là thành phần của Windows cho phép chạy môi trường Linux trên Windows. Docker Desktop trên Windows có thể dùng WSL 2 làm backend để chạy Linux container.

Hai lệnh kiểm tra:

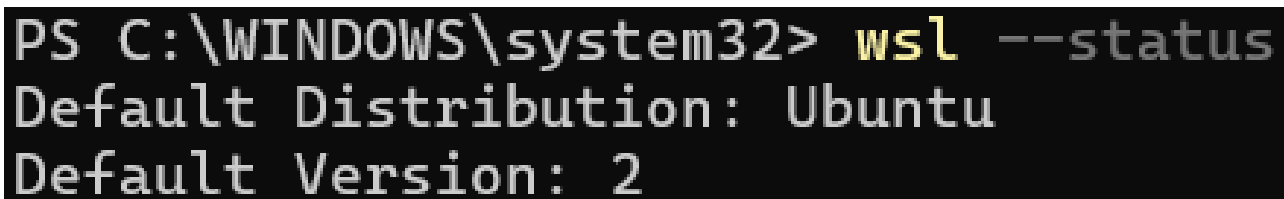
```
wsl --version
```

```
wsl --status
```



```
PS C:\WINDOWS\system32> wsl --version
WSL version: 2.7.14.0
Kernel version: 6.18.33.2-2
WSLg version: 1.0.73.2
MSRDC version: 1.2.7214
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.9457
```

Hình 1. Kết quả wsl --version trên máy học viên.



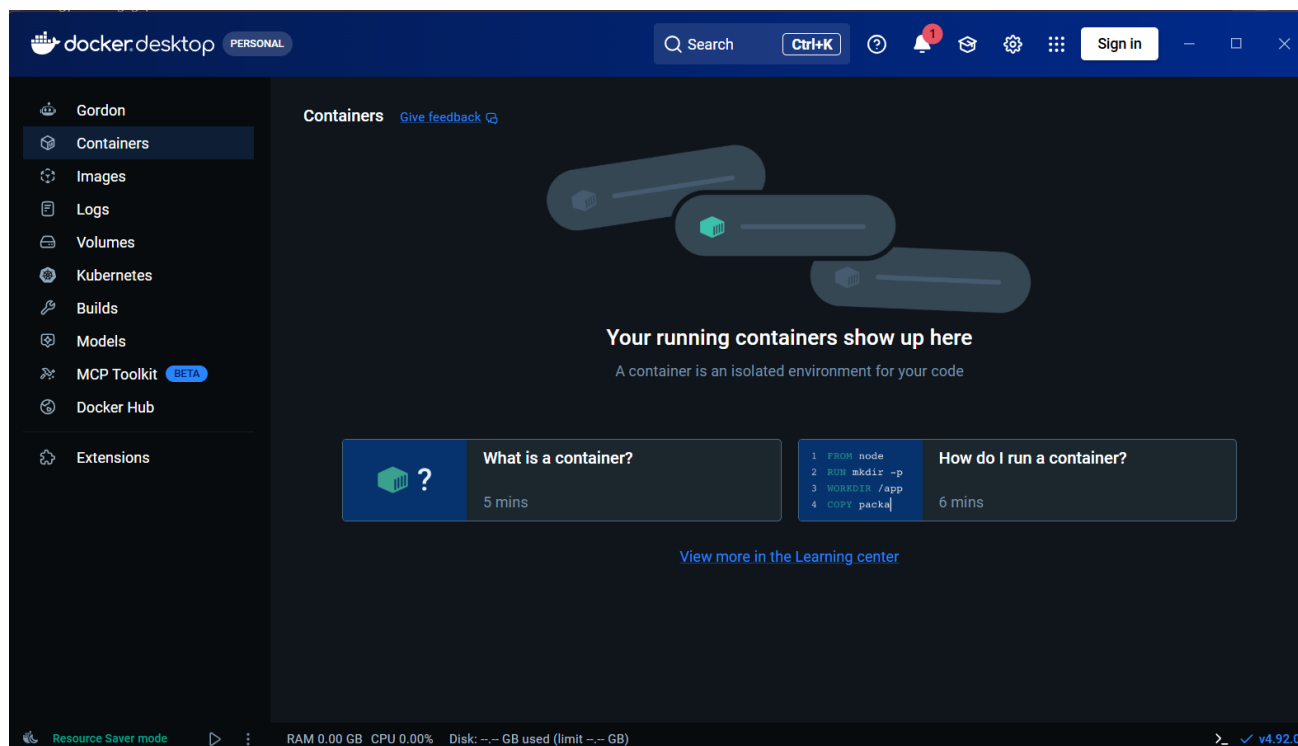
```
PS C:\WINDOWS\system32> wsl --status
Default Distribution: Ubuntu
Default Version: 2
```

Hình 2. WSL đang dùng Default Version: 2 và distro mặc định là Ubuntu.

Kết quả trên cho thấy WSL 2 đã hoạt động. Điều quan trọng là dòng Default Version: 2. Ubuntu đã có sẵn cũng không gây trở ngại; không cần cài lại WSL.

## 2. Khởi động Docker Desktop

Sau khi cài Docker Desktop, mở ứng dụng và chờ Docker Engine khởi động. Giao diện Docker Desktop cung cấp các khu vực quản lý Container, Image, Volume, Build và các thành phần khác.



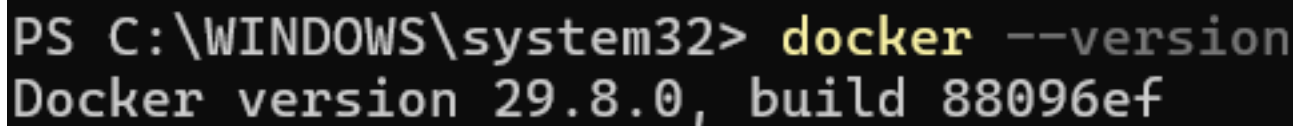
Hình 3. Docker Desktop đã khởi động thành công; chưa có container đang chạy.

Lưu ý về ổ đĩa: nếu mục tiêu là hạn chế Docker chiếm ổ C, nên đặt Docker Data/Disk image ở ổ E. Phần này chứa image, container, volume và dữ liệu có thể tăng dung lượng theo thời gian.

### 3. Kiểm tra Docker CLI

Docker CLI là chương trình dòng lệnh mà người dùng gọi bằng lệnh docker. Để kiểm tra CLI đã được cài và PowerShell nhận được lệnh Docker:

```
docker --version
```



```
PS C:\WINDOWS\system32> docker --version
Docker version 29.8.0, build 88096ef
```

Hình 4. docker --version trả về Docker version 29.8.0.

Kết luận: Docker CLI đã được cài thành công.

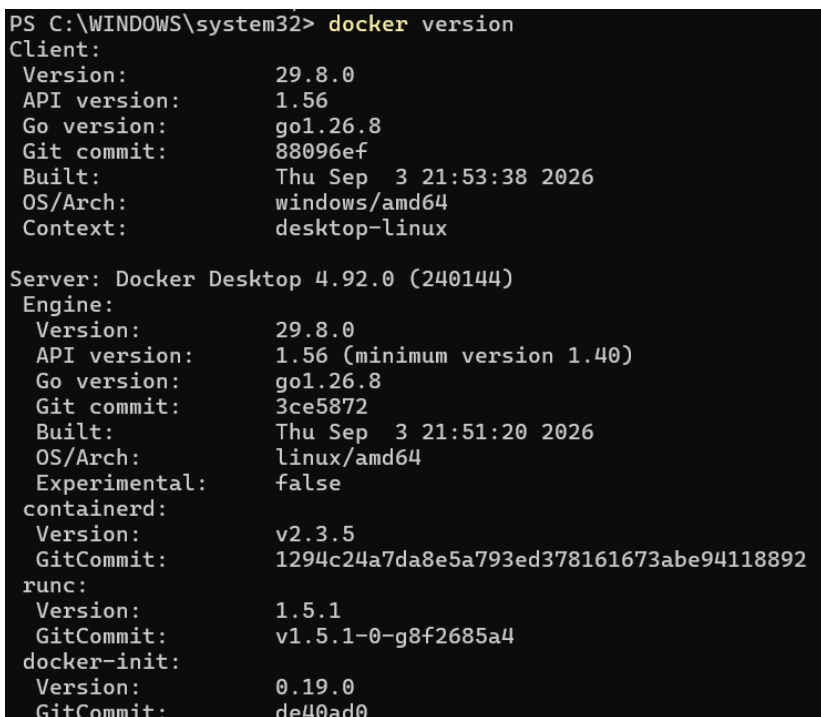
Điểm cần nhớ: docker --version chủ yếu kiểm tra Docker CLI. Lệnh này chưa đủ để chứng minh Docker Engine đang hoạt động.

### 4. Kiểm tra Docker Engine

Tiếp tục chạy:

```
docker version
```

Lệnh này hiển thị cả phần Client và Server. Trong bài học này: Client tương ứng Docker CLI; Server tương ứng Docker Engine/Docker Daemon phía nhận và xử lý yêu cầu.



```
PS C:\WINDOWS\system32> docker version
Client:
 Version:           29.8.0
 API version:       1.56
 Go version:        go1.26.8
 Git commit:        88096ef
 Built:             Thu Sep  3 21:53:38 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.92.0 (240144)
 Engine:
  Version:          29.8.0
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.8
  Git commit:       3ce5872
  Built:            Thu Sep  3 21:51:20 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.3.5
  GitCommit:        1294c24a7da8e5a793ed378161673abe94118892
 runc:
  Version:          1.5.1
  GitCommit:        v1.5.1-0-g8f2685a4
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

Hình 5. docker version có cả Client và Server, xác nhận CLI giao tiếp được với Docker Engine.

## 5. Kiểm tra môi trường Docker bằng docker info

Lệnh sau yêu cầu Docker Engine trả về thông tin chi tiết về môi trường hiện tại:

docker info

Ở giai đoạn này không cần hiểu toàn bộ các mục như overlayfs, cgroup, containerd hay runc. Chỉ cần xác nhận phần Server xuất hiện bình thường và không có lỗi kết nối Engine.

```
PS C:\WINDOWS\system32> docker info
Client:
Version: 20.8.0
Context: desktop-linux
Debug Mode: false
Plugins:
agent: Docker AI Agent Runner (Docker Inc.)
       version: v1.30.0
       path: C:\Program Files\Docker\cli-plugins\docker-agent.exe
       ai: Docker AI Agent - Ask DevOps (Docker Inc.)
       version: v1.31.0
       path: C:\Program Files\Docker\cli-plugins\docker-ai.exe
       buildx: Docker Buildx (Docker Inc.)
       version: v0.7.1
       path: C:\Program Files\Docker\cli-plugins\docker-buildx.exe
       compose: Docker Compose (Docker Inc.)
       version: v0.5.1
       path: C:\Program Files\Docker\cli-plugins\docker-compose.exe
       debug: Get a shell into any image or container (Docker Inc.)
       version: 0.0.0
       path: C:\Program Files\Docker\cli-plugins\docker-debug.exe
       desktop: Docker Desktop Commands (Docker Inc.)
       version: v0.4.0
       path: C:\Program Files\Docker\cli-plugins\docker-desktop.exe
       dhi: CLI for managing Docker Hardened Images (Docker Inc.)
       version: v0.0.0
       path: C:\Program Files\Docker\cli-plugins\docker-dhi.exe
       extension: Manage Docker extensions (Docker Inc.)
       version: v0.2.1
       path: C:\Program Files\Docker\cli-plugins\docker-extension.exe
       init: Creates Docker-related starter files for your project (Docker Inc.)
       version: v1.0.0
       path: C:\Program Files\Docker\cli-plugins\docker-init.exe
       mcp: Docker MCP Plugin (Docker Inc.)
       version: v0.0.1
       path: C:\Program Files\Docker\cli-plugins\docker-mcp.exe
       model: Docker Model Runner (Docker Inc.)
       version: v1.0.0
       path: C:\Program Files\Docker\cli-plugins\docker-model.exe
       offload: Docker Offload (Docker Inc.)
       version: v0.6.27
       path: C:\Program Files\Docker\cli-plugins\docker-offload.exe
       pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
       version: v0.0.1
       path: C:\Program Files\Docker\cli-plugins\docker-pass.exe
       sandbox: Docker Sandbox is deprecated, use Docker Sandboxes instead (Docker Inc.)
       version: v0.11.0
       path: C:\Program Files\Docker\cli-plugins\docker-sandbox.exe
       scout: Docker Scout (Docker Inc.)
       version: v1.20.0
       path: C:\Program Files\Docker\cli-plugins\docker-scout.exe
Server:
Containers: 0
Running: 0
Paused: 0
Stopped: 0
Images: 0
Server Version: 20.8.0
Storage Driver: overlayfs
```

Hình 6. Phần đầu kết quả docker info: Client, plugin và Server.

```
WARNING: 'docker sandbox' is deprecated, use Docker Sandboxes instead (Docker Inc.)
Version: v0.11.0
Path: C:\Program Files\Docker\cli-plugins\docker-sandbox.exe
scout: Docker Scout (Docker Inc.)
Version: v1.20.0
Path: C:\Program Files\Docker\cli-plugins\docker-scout.exe
Server:
Containers: 0
Running: 0
Paused: 0
Stopped: 0
Images: 0
Server Version: 20.8.0
Storage Driver: overlayfs
Driver Type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
Plugins:
Volume: local
Network: bridge host ipvlan macvlan null overlay
Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
/etc/cdi
/etc/kubernetes
Discovered Devices:
cdi: docker.com/powershpc
Swarm: inactive
Runtimes: io.containerd.runc.v2 nvidia runc
Default Runtime: runc
Init Binary: docker-init
containerd version: 120ac2647d8e5793ed378161073abe94118892
runc version: v1.0.0-gf2808ae
init version: de40ad6
Security Options:
seccomp
Profile: builtin
cgroups:
kernel version: 6.18.33.2-microsoft-standard-WSL2
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 12
Total Memory: 3.78GiB
Name: docker-desktop
ID: cf783008-99a7-4a0f-b3cf-33bd77f712d0
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure: registries
hubproxy.docker.internal:5555
ip: 0.0.0.0
ip6: ::1/128
Live Restore Enabled: false
Firewall: iptables
```

Hình 7. docker info cho thấy Docker Desktop đang dùng Linux/WSL 2, Containers: 0 và Images: 0 trước khi chạy hello-world.

Trong ảnh có các thông tin đáng chú ý: Kernel Version ... WSL2, Operating System: Docker Desktop, OSType: linux. Điều này cho thấy Docker Engine đang vận hành môi trường Linux container thông qua Docker Desktop/WSL 2.

## 6. Chạy container đầu tiên: hello-world

Thực hiện:

```
docker run hello-world
```

Lần chạy đầu tiên, máy chưa có image hello-world:latest, vì vậy Docker thông báo không tìm thấy image ở local, sau đó tự tải image về từ registry và tạo container từ image đó.

```
PS C:\WINDOWS\system32> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Hình 8. hello-world được pull, container được tạo và chương trình in ra “Hello from Docker!”.

Không phải lỗi: dòng Unable to find image 'hello-world:latest' locally chỉ nói rằng image chưa tồn tại trên máy. Docker sau đó tự pull image về.

## 7. Phân tích chính xác luồng docker run hello-world

| Bước | Điều xảy ra                                                  |
|------|--------------------------------------------------------------|
| 1    | Người dùng nhập docker run hello-world trong PowerShell.     |
| 2    | Docker CLI gửi yêu cầu tới Docker Engine.                    |
| 3    | Docker Engine kiểm tra image hello-world:latest ở local.     |
| 4    | Image chưa có nên Docker tải image từ registry (Docker Hub). |
| 5    | Docker Engine tạo một container từ image hello-world.        |
| 6    | Container khởi động process chính bên trong image.           |
| 7    | Process in ra thông báo Hello from Docker!.                  |
| 8    | Process kết thúc; container chuyển sang trạng thái dừng.     |

### Quan hệ cốt lõi:

Docker Hub / Registry  
↓ pull  
Image  
↓ docker run  
Container  
↓  
Process  
↓ kết thúc  
Container dừng

Điểm quan trọng: Docker tải Image về. Container không phải thứ được tải từ Docker Hub; container được Docker Engine tạo ra từ image trên máy local.

## 8. Liên hệ lại các khái niệm của Bài 1

| Khái niệm     | Đã quan sát ở Bài 2                                                     |
|---------------|-------------------------------------------------------------------------|
| Docker CLI    | Lệnh docker --version và phần Client trong docker version.              |
| Docker Engine | Phần Server trong docker version và thông tin Server trong docker info. |
| Image         | hello-world:latest được pull về khi local chưa có.                      |
| Container     | Được tạo từ image hello-world khi chạy docker run.                      |
| Process chính | Chương trình bên trong container in thông báo rồi kết thúc.             |
| WSL 2         | Kernel/Backend Linux được Docker Desktop sử dụng trên Windows.          |

## 9. Câu hỏi tự kiểm tra và kết quả

1. docker --version chủ yếu kiểm tra gì?

Docker CLI. Đây là điểm cần sửa từ câu trả lời ban đầu.

2. Client và Server trong docker version là gì?

Client = Docker CLI; Server = Docker Engine/Docker Daemon.

3. Vì sao docker run hello-world phải tải dữ liệu?

Vì image hello-world chưa có ở local.

4. Thứ tải từ Docker Hub về là gì?

Image, không phải Container.

5. Vì sao hello-world chạy xong thì container dừng?

Vì process chính bên trong container đã thực thi xong và kết thúc.

## 10. Kết luận Bài 2

Máy đã đáp ứng các điều kiện thực hành Docker: WSL 2 hoạt động, Docker Desktop khởi động, Docker CLI giao tiếp được với Docker Engine, docker info hoạt động và container hello-world chạy thành công.

### Sau bài này, cần nắm chắc hai chuỗi logic sau:

docker --version → kiểm tra Docker CLI

docker version → kiểm tra Client + Server (CLI + Engine)

Image → được tải/lưu ở local

Container → được tạo từ Image để chạy process

Bài tiếp theo: Chương 1 - Bài 3: làm việc với Image và vòng đời Container bằng docker pull, docker run, docker ps, docker stop, docker start, docker rm, docker images và docker rmi.